

CONTROLLER PROGRAMMING

Timer0 & Timer2 (8-bit Timer/Counter) Programming

01. Timer1 Programming using Interrupt (From Lecture):-

```
void init_port();  
void init_timer();
```

```
void setup() {
```

```
volatile long i;
int j=0;
init_port();
init_timer();
Serial.begin(9600);
while(1)
{
    for(i=0; i<500000; i++);
    Serial.println(j,DEC);
    j++;
}
}
```

```
void init_timer(){
    volatile char *Timer1_TCCR1A = 0x80; //
    Timer/Counter 1 Control Register A
    volatile char *Timer1_TCCR1B = 0x81; //
    Timer/Counter 1 Control Register B
    volatile short *Timer1_TCNT1 = 0x84; //
    Timer/Counter 1 Register
    volatile short *Timer1_OCR1A = 0x88; // Output
    Compare Register 1 A
    volatile char *Timer1_TIMSK1 = 0x6F; //
    Timer/Counter 1 Interrupt Mask register
}
```

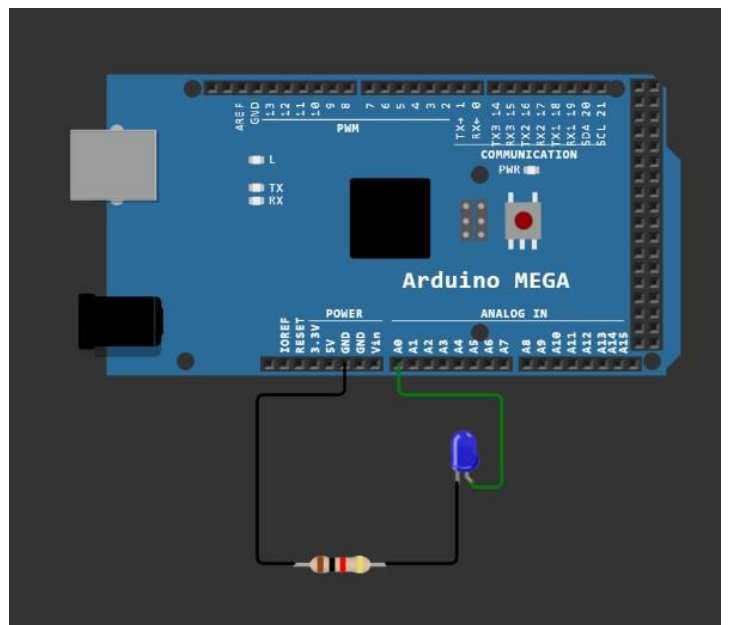
```
*Timer1_TCCR1A = 0;
// *Timer1_TCCR1B = 0;
*Timer1_TCCR1B = 0x0c; // TCCR1A & TCCR1B are
assigned such that the timer 1 acts as CTC mode and
clk/256 prescaler mode
*Timer1_TCNT1 = 0; // count 0 to OCR1A value

*Timer1_OCR1A = 60000; // our required value
*Timer1_TIMSK1 = 0x02; // set the interrupt mask
register OCIE1A bit
}
```

```
void init_port(){
    volatile char *portf_dir = (volatile char *) 0x30;
    *portf_dir = 0x01;
}

ISR(TIMER1_COMPA_vect){
    volatile char *portf_data = (volatile char *) 0x31;
    *portf_data ^= 0x01;
}

void loop() {}
```



02. Timer1 & Timer2 Programming using Interrupt (From Lecture):-

```
void init_port();  
void init_timer1();  
void init_timer3();
```

```
volatile char *portf_data;  
volatile char *portk_data;
```

```
void setup() {
```

```
    volatile long i;  
    int j=0;  
    init_port();  
    init_timer1();  
    init_timer3();  
    Serial.begin(9600);  
    while(1)  
    {  
        for(i=0; i<500000; i++);  
        Serial.println(j,DEC);  
        j++;  
    }  
}
```

```
void init_timer3(){                // By using Normal  
mode
```

```
    volatile char *Timer3_TCCR3A = 0x90;  
    volatile char *Timer3_TCCR3B = 0x91;  
    volatile short *Timer3_TCNT3 = 0x94;  
    volatile char *Timer3_TIFR3 = 0x71;
```

```
    *Timer3_TCCR3A = 0;  
    *Timer3_TCCR3B = 0x04;  
    *Timer3_TCNT3 = 0;  
    *Timer3_TIFR3 = 1;  
}
```

```
void init_timer1(){                // By using CTC mode
```

```
    volatile char *Timer1_TCCR1A = 0x80;  
    volatile char *Timer1_TCCR1B = 0x81;  
    volatile short *Timer1_TCNT1 = 0x84;  
    volatile short *Timer1_OCR1A = 0x88;  
    volatile char *Timer1_TIMSK1 = 0x6F;
```

```
*Timer1_TCCR1A = 0;  
/*Timer1_TCCR1B = 0;  
*Timer1_TCCR1B = 0x0c; // TCCR1A & TCCR1B are  
assigned such that the timer 1 acts as CTC mode and  
clk/256 prescaler mode  
*Timer1_TCNT1 = 0; // count 0 to OCR1A value
```

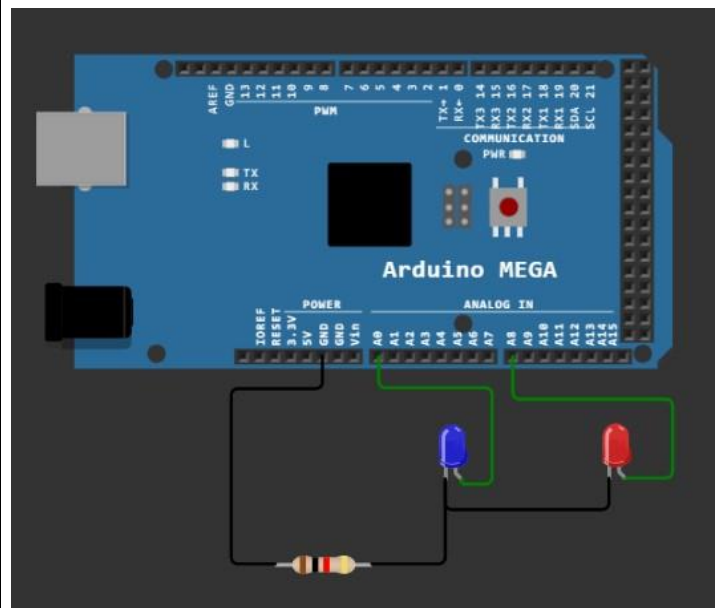
```
*Timer1_OCR1A = 8000; // our required value  
*Timer1_TIMSK1 = 0x02; // set the interrupt mask  
register OCIE1A bit  
}
```

```
void init_port(){  
    volatile char *portf_dir = 0x30, *portk_dir = 0x107;  
    *portf_dir = 0x01; *portk_dir = 1;  
    portf_data = 0x31; portk_data = 0x108;  
}
```

```
ISR(TIMER1_COMPA_vect){  
    *portf_data ^= 0x01;  
}
```

```
ISR(TIMER3_OVF_vect){  
    *portk_data ^= 0x01;  
}
```

```
void loop() {}
```



03. Timer0 programming in Normal Mode:-

(Another program of Normal Mode using Interrupt are at the last of the 8-bit Timer/Counter)

```
uint8_t i=0;

void setup() {
  Serial.begin(9600);

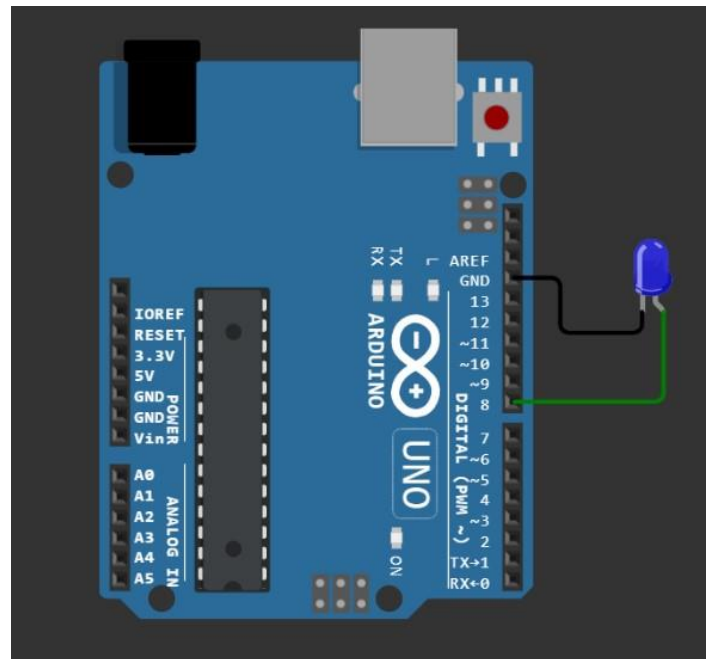
  DDRB = 1;

  TCCR0B = 5; // 5 for 1024 prescaler
  TCNT0 = 0;

  while(1){
    if(TCNT0>=255){
      i++;
      if(i%20==0){
        PORTB ^= 1;
        Serial.println(i);
      }
      TCNT0=0;
    }
  }
}

void loop() {}
```

From 03, all program written by myself



04. Timer0 programming in CTC Mode, OC0A pin toggling on compare match:-

```
uint8_t count_compare;

void setup() {
  Serial.begin(9600);
  DDRD |= 64; // Set pin6 as Output

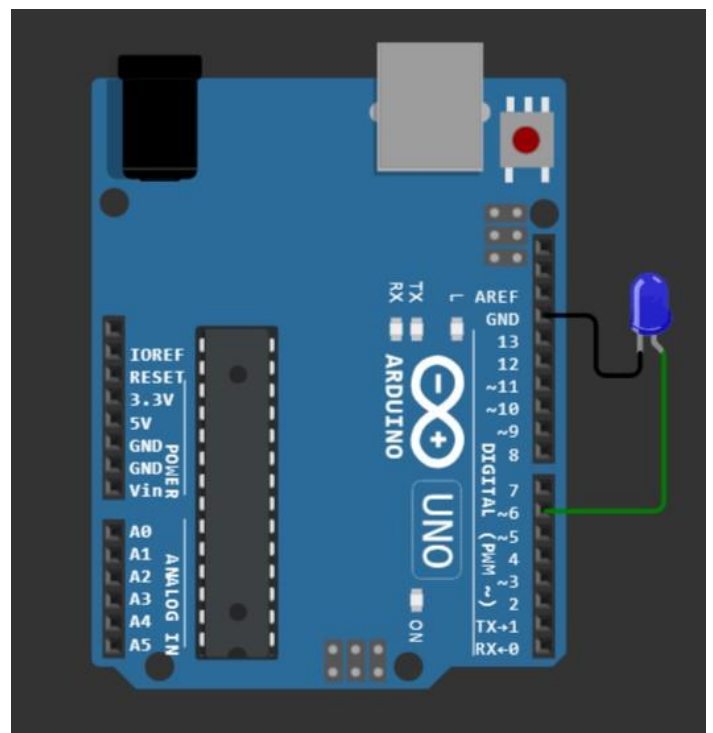
  TCCR0A = 64+2; // 64 for toggling OC0A pin on
  compare match & 2 for CTC mode
  TCCR0B = 5; // 5 for 1024 prescaler
  TCNT0 = 0;
  OCR0A = 255;

  while(1){
    if(TCNT0==250){
      count_compare++;
      Serial.println(count_compare);
    }
  }
}

void loop() {}

// Output > Toggled the OC0A pin (i.e., Arduino pin 6)
```

For OC0A Pin Operation



05. Timer0 programming in CTC Mode, OC0B pin toggling on compare match:-

```
uint8_t count_compare;

void setup() {
  Serial.begin(9600);
  DDRD |= 32; // Set pin5 as Output

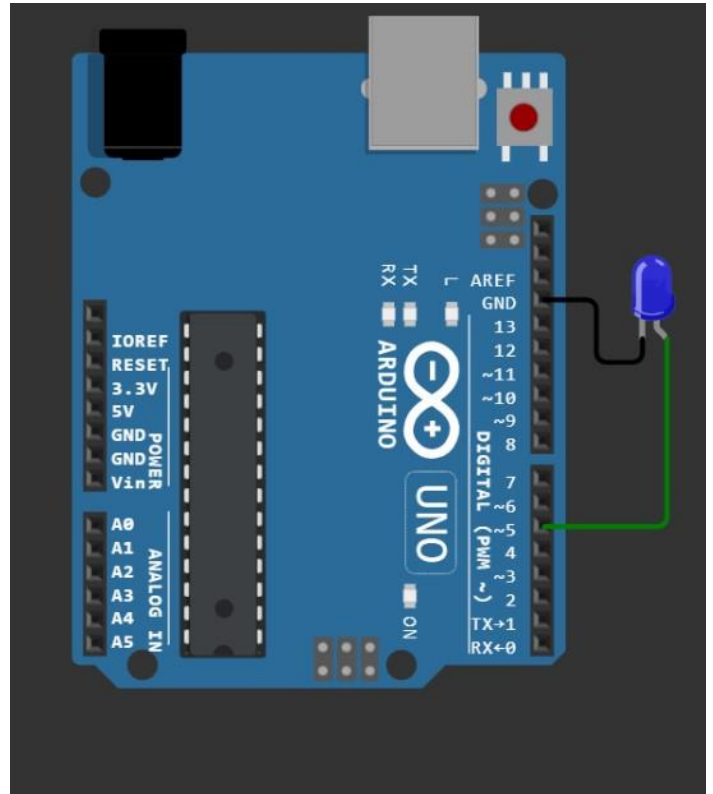
  TCCR0A = 2 + 16; // 16 for toggling OC0B pin on
  compare match & 2 for CTC mode
  TCCR0B = 5; // 5 for 1024 prescaler
  TCNT0 = 0;
  OCR0A = 255;

  while(1){
    if(TCNT0==250){
      count_compare++;
      Serial.println(count_compare);
    }
  }
}

void loop() {}

// Output > Toggled the OC0B pin (i.e., Arduino pin 5)
```

For OC0B Pin Operation



06. Timer0 programming in Fast PWM (Inverting) Mode for OC0A pin:-

```
void setup() {
  DDRD |= 64; // Set pin6 as Output

  TCCR0A = 128+64+2+1; // 128+64 for inverting mode
  PWM & 2+1 for Fast PWM generation
  // Inverting mode means set OC0A on compare
  match and clear OC0A at bottom
  TCCR0B = 5; // 5 for 1024 prescaler
  TCNT0 = 0;
  OCR0A = 255; // According to the value of OCR0A,
  output will change

  while(1){}
}

void loop() {}

// Output > PWM output at OC0A pin (i.e., Arduino pin
6)
```

07. Timer0 programming in Fast PWM (Non - Inverting) Mode for OC0A pin:-

```
void setup() {
  DDRD |= 64; // Set pin6 as Output

  TCCR0A = 128+2+1; // 128 for non-inverting mode
  PWM & 2+1 for Fast PWM generation
  // Non-inverting mode means clear OC0A on
  compare match and set OC0A at bottom
  TCCR0B = 5; // 5 for 1024 prescaler
  TCNT0 = 0;
  OCR0A = 25; // According to the value of OCR0A,
  output will change

  while(1){}
}

void loop() {}

// Output > PWM output at OC0A pin (i.e., Arduino pin
6)
```

08. Timer0 programming in Fast PWM (Inverting) Mode for OC0B pin:-

```
void setup() {  
  DDRD |= 32; // Set pin5 as Output  
  
  TCCR0A = 32+16+2+1; // 32+16 for inverting mode  
  PWM & 2+1 for Fast PWM generation  
    // Inverting mode means set OC0B on compare  
  match and clear OC0B at bottom  
  TCCR0B = 5; // 5 for 1024 prescaler  
  TCNT0 = 0;  
  OCR0B = 255; // According to the value of OCR0B,  
  output will change  
  
  while(1){}  
}  
  
void loop() {}  
  
// Output > PWM output at OC0B pin (i.e., Arduino pin  
5)
```

09. Timer0 programming in Fast PWM (Non - Inverting) Mode for OC0B pin:-

```
void setup() {  
  DDRD |= 32; // Set pin5 as Output  
  
  TCCR0A = 32+2+1; // 32 for non-inverting mode PWM  
  & 2+1 for Fast PWM generation  
    // Non-inverting mode means clear OC0B on  
  compare match and set OC0B at bottom  
  TCCR0B = 5; // 5 for 1024 prescaler  
  TCNT0 = 0;  
  OCR0B = 25; // According to the value of OCR0B,  
  output will change  
  
  while(1){}  
}  
  
void loop() {}  
  
// Output > PWM output at OC0B pin (i.e., Arduino pin  
5)
```

10. Timer0 programming in Phase Correct PWM Mode (COM0A1=1 & COM0A0=1) for OC0A pin:-

```
void setup() {  
  DDRD |= 64; // Set pin6 as Output  
  
  TCCR0A = 128+64+1; // 128+64 for set OC0A on  
  compare match when up-counting  
  // and clear OC0A on compare match when down-  
  counting & 1 for Phase Correct PWM generation  
  
  TCCR0B = 5; // 5 for 1024 prescaler  
  TCNT0 = 0;  
  OCR0A = 250; // According to the value of OCR0A,  
  output will change  
  
  while(1){}  
}  
  
void loop() {}  
  
// Output > PWM output at OC0A pin (i.e., Arduino pin  
6)
```

11. Timer0 programming in Phase Correct PWM Mode (COM0A1=1 & COM0A0=0) for OC0A pin:-

```
void setup() {  
  DDRD |= 64; // Set pin6 as Output  
  
  TCCR0A = 128+1; // 128 for clear OC0A on compare  
  match when up-counting  
  // and set OC0A on compare match when down-  
  counting & 1 for Phase Correct PWM generation  
  
  TCCR0B = 5; // 5 for 1024 prescaler  
  TCNT0 = 0;  
  OCR0A = 10; // According to the value of OCR0A,  
  output will change  
  
  while(1){}  
}  
  
void loop() {}  
  
// Output > PWM output at OC0A pin (i.e., Arduino pin  
6)
```

12. Timer0 programming in Phase Correct PWM Mode (COM0B1=1 & COM0B0=1) for OC0B pin:-

```
void setup() {  
  DDRD |= 32; // Set pin5 as Output  
  
  TCCR0A = 32+16+1; // 32+16 for set OC0B on  
  compare match when up-counting  
  // and clear OC0B on compare match when down-  
  counting & 1 for Phase Correct PWM generation  
  
  TCCR0B = 5; // 5 for 1024 prescaler  
  TCNT0 = 0;  
  OCR0B = 250; // According to the value of OCR0B,  
  output will change  
  
  while(1){}  
}  
  
void loop() {}  
  
// Output > PWM output at OC0B pin (i.e., Arduino  
pin 5)
```

13. Timer0 programming in Phase Correct PWM Mode (COM0B1=1 & COM0B0=0) for OC0B pin:-

```
void setup() {  
  DDRD |= 32; // Set pin5 as Output  
  
  TCCR0A = 32+1; // 32 for clear OC0B on compare  
  match when up-counting  
  // and set OC0B on compare match when down-  
  counting & 1 for Phase Correct PWM generation  
  
  TCCR0B = 5; // 5 for 1024 prescaler  
  TCNT0 = 0;  
  OCR0B = 10; // According to the value of OCR0B,  
  output will change  
  
  while(1){}  
}  
  
void loop() {}  
  
// Output > PWM output at OC0B pin (i.e., Arduino  
pin 5)
```

14. Timer2 programming, Normal Mode, using Overflow Interrupt. (Timer0 Overflow Interrupt not working) :-

```
void setup() {  
  Serial.begin(9600);  
  
  TCCR2A = 0; // Normal Mode  
  TCCR2B = 5; // 5 for 1024 prescaler  
  TIMSK2 = 1; // enable the Timer2 Overflow Interrupt  
  TCNT2 = 0;  
  
  while(1){}  
}  
  
ISR(TIMER2_OVF_vect){  
  Serial.println("hello world");  
}  
  
void loop() {}
```

15. Timer0 programming, CTC Mode Compare Match A:-

```
void setup() {  
  Serial.begin(9600);  
  
  TCCR0A = 2; // CTC Mode  
  TCCR0B = 5; // 5 for 1024 prescaler  
  TIMSK0 = 2; // enable the Timer0 Output Compare  
  Match A Interrupt  
  TCNT0 = 0;  
  OCR0A = 250;  
  
  while(1){}  
}  
  
ISR(TIMER0_COMPA_vect){  
  Serial.println("hello world");  
}  
  
void loop() {}
```

16. Timer0 programming, CTC Mode Compare Match B:-

```
void setup() {
  Serial.begin(9600);

  TCCR0A = 2; // CTC Mode
  TCCR0B = 5; // 5 for 1024 prescaler
  TIMSK0 = 4; //enable the Timer0 Output Compare
  Match B Interrupt
  TCNT0 = 0;
  OCR0A = 250; // if I use OCR0B = 250; then not
  working, may be it is the problem of the wokwi
  simulator

  while(1){}
}

ISR(TIMERO_COMPB_vect){
  Serial.println("hello world");
}

void loop() {}
```

17. Use of TIFR Register (self) :-

```
volatile long i=0;

void setup() {
  Serial.begin(9600);

  TCCR0A = 2; // CTC Mode
  TCCR0B = 5; // 5 for 1024 prescaler
  TIMSK0 = 2; // enable the Timer0 Output Compare
  Match A Interrupt
  TCNT0 = 0;
  OCR0A = 250;

  while(1){
    if(i%100==0)
      {Serial.println("hello world"); i++;}
  }
}

ISR(TIMERO_COMPA_vect){
  if((TIFR0 & 2)==0)
    i++;
}

void loop() {}
```

18. Brightness Control by using PWM Mode (self-written-code) :-

```
volatile uint8_t count_flag, brightness=0;

void setup() {
  DDRD |= 64; // Set pin6 as Output

  TCCR0A = 128+64+2+1; // 128+64 for inverting mode
  PWM & 2+1 for Fast PWM generation
  // Inverting mode means set OCOA on compare
  match and clear OCOA at bottom
  TCCR0B = 5; // 5 for 1024 prescaler
  TCNT0 = 0;
  OCR0A = 255; // According to the value of OCR0A,
  output will change
  TIMSK0 = 2; // Enable the Timer/Counter0 Output
  Compare Match A Interrupt
  while(1){}
}

ISR(TIMERO_COMPA_vect){
  if(brightness==0) count_flag=0;
  if(brightness>250) count_flag=1;
  if(count_flag==0) brightness+=3;
  else brightness-=3;
  OCR0A = brightness;
}

void loop() {}
```

19. Counter Programming by using Timer/Counter0 :-

```
void setup() {
  Serial.begin(9600);

  DDRB = 1;
  TCCR0A = 0; // Normal Mode
  TCCR0B = 7; // 7 for selecting External Clock source on
  T0 pin (Rising Edge)
  TCNT0 = 0;

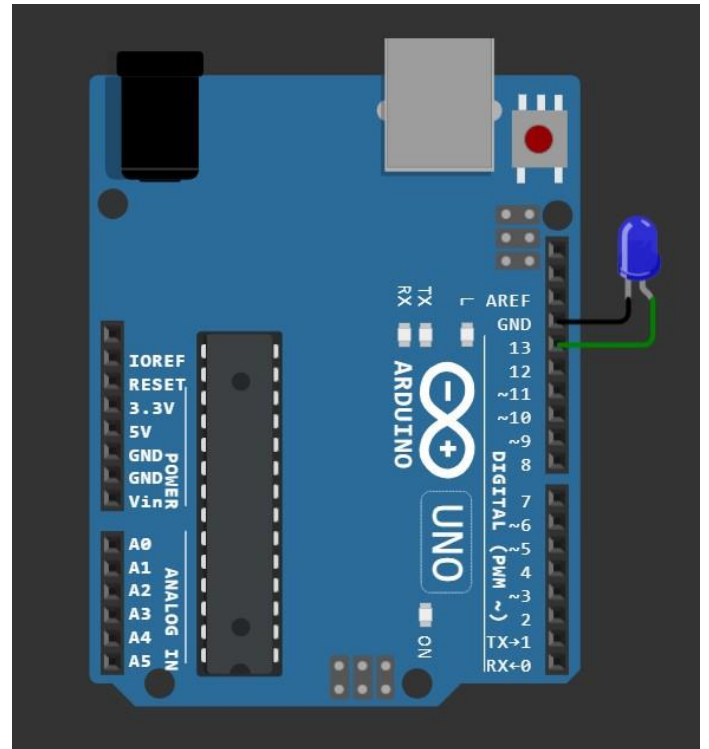
  while(1){
    if(PORTB & 1) Serial.println(TCNT0);
    PORTB ^= 1;
    for(volatile long i=0; i<5000; i++);
    if(TCNT0>250) TCNT0=0;
  }
}

void loop() {}
```


Timer1 (16-bit Timer/Counter) Programming

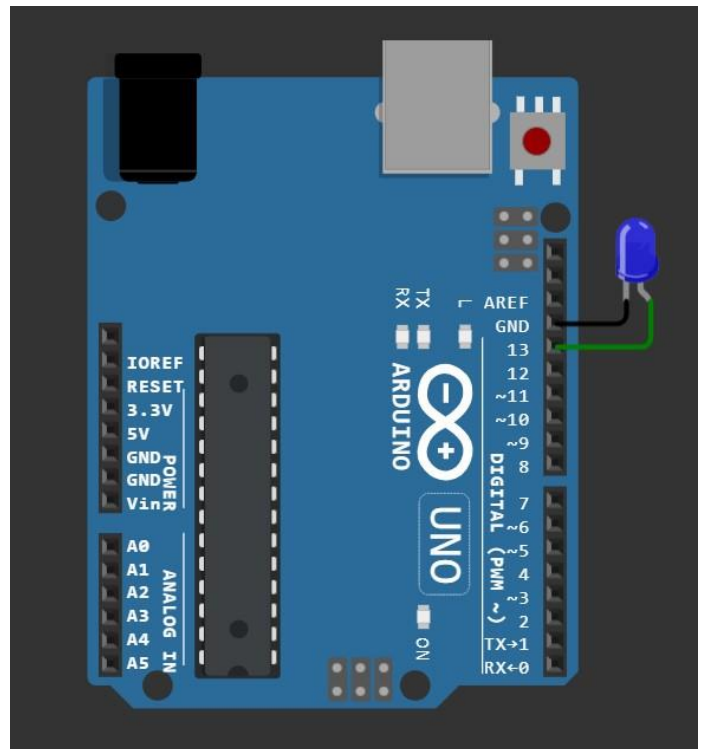
01. Timer1 programming in Normal Mode:-

```
void setup() {  
  
    DDRB = 32; // PORTB 5th bit set as Output  
  
    TCCR1A = 0; // 0 for Normal Mode  
    TCCR1B = 5; // 5 for selecting 1024 prescaler  
    TCNT1 = 0;  
  
    while(1){  
  
        if(TCNT1>=10000){  
            PORTB ^= 32;  
            TCNT1=0;  
        }  
    }  
}  
  
void loop() {}
```



02. Timer1 programming, Normal Mode, using Overflow Interrupt :-

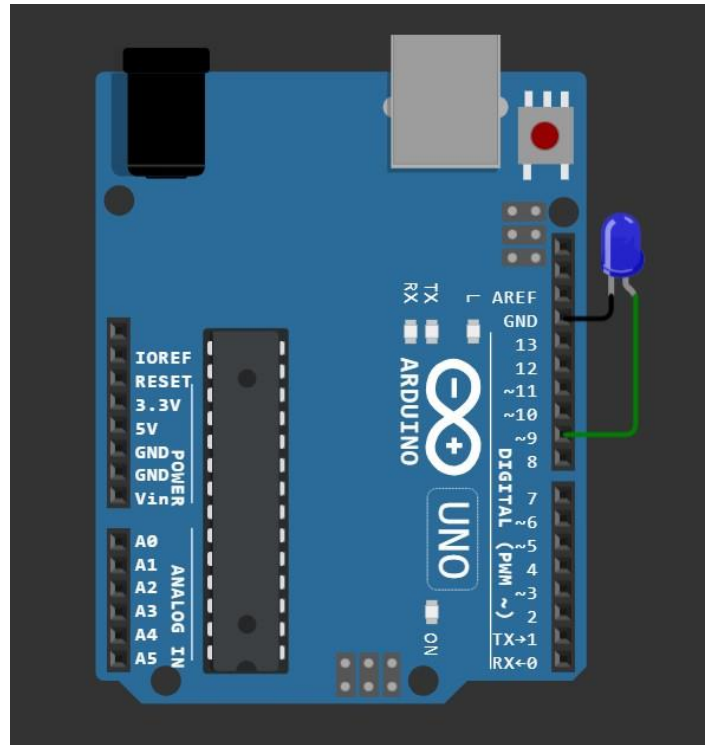
```
void setup() {  
  
    DDRB = 32; // PORTB 5th bit set as Output  
  
    TCCR1A = 0; // 0 for Normal Mode  
    TCCR1B = 3; // 3 for selecting 64 prescaler  
    TIMSK1 = 1; // for setting the Overflow Flag  
    TCNT1 = 0;  
  
    while(1){}  
}  
  
ISR(TIMER1_OVF_vect){  
    PORTB ^= 32;  
}  
  
void loop() {}
```



03. Timer1 programming in CTC Mode, OC1A pin toggling on compare match:-

```
void setup() {  
  
    DDRB = 2; // PORTB 1st bit set as Output  
  
    TCCR1A = 64; // 64 for toggling OC1A pin on  
compare match  
    TCCR1B = 5 + 8; // 5 for selecting 1024 prescaler & 8  
for selecting CTC Mode  
    OCR1A = 5000;  
    TCNT1 = 0;  
  
    while(1){}  
}  
  
void loop() {}
```

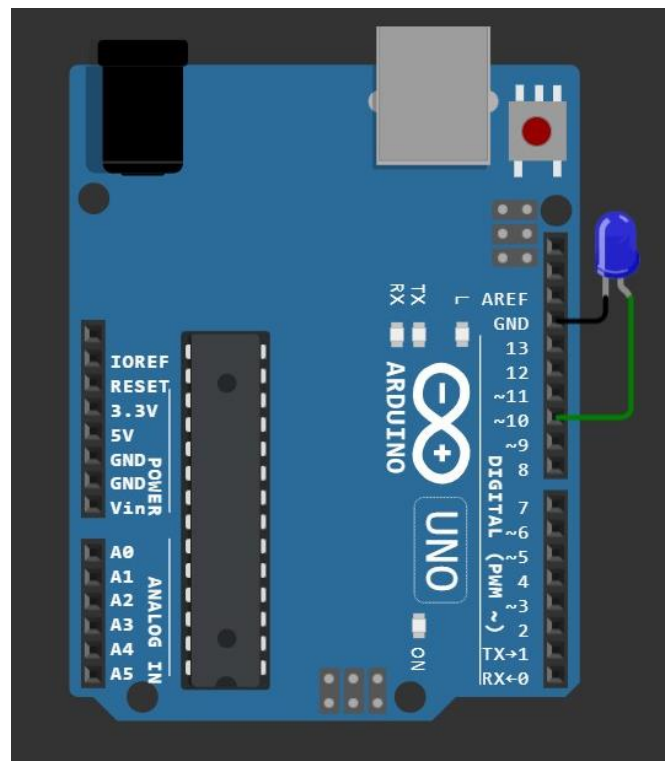
For OC1A Pin Operation



04. Timer1 programming in CTC Mode, OC1B pin toggling on compare match:-

```
void setup() {  
  
    DDRB = 4; // PORTB 2nd bit set as Output  
  
    TCCR1A = 16; // 16 for toggling OC1B pin on  
compare match  
    TCCR1B = 5 + 8; // 5 for selecting 1024 prescaler & 8  
for selecting CTC Mode  
    OCR1A = 5000;  
    TCNT1 = 0;  
  
    while(1){}  
}  
  
void loop() {}
```

For OC1B Pin Operation



05. Timer1 programming in Fast PWM (10-bit & Inverting) Mode for OC1A pin:-

```
void setup() {  
  
    DDRB = 2; // PORTB 1st bit set as Output  
  
    TCCR1A = 128 + 64 + 2 + 1; // 128 + 64 for Set OC1A  
on compare match,  
        //clear OC1A at BOTTOM (inverting  
mode)  
        // and 2 + 1 for selecting Fast PWM, 10-bit  
    TCCR1B = 4 + 8; // 4 for selecting 256 prescaler & 8  
for selecting Fast PWM, 10-bit  
    OCR1A = 1000;  
    TCNT1 = 0;  
  
    while(1){}  
}  
  
void loop() {}
```

06. Timer1 programming in Fast PWM (10-bit Non - Inverting) Mode for OC1A pin:-

```
void setup() {  
  
    DDRB = 2; // PORTB 1st bit set as Output  
  
    TCCR1A = 128 + 2 + 1; // 128 for clear OC1A on  
compare match,  
        //set OC1A at BOTTOM (Non-  
inverting mode)  
        // and 2 + 1 for selecting Fast PWM, 10-bit  
    TCCR1B = 4 + 8; // 4 for selecting 256 prescaler & 8  
for selecting Fast PWM, 10-bit  
    OCR1A = 10;  
    TCNT1 = 0;  
  
    while(1){}  
}  
  
void loop() {}
```

07. Timer1 programming in Fast PWM (10-bit Inverting) Mode for OC1B pin:-

```
void setup() {  
  
    DDRB = 4; // PORTB 2nd bit set as Output  
  
    TCCR1A = 32 + 16 + 2 + 1; // 32 + 16 for set OC1B on  
compare match, clear OC1B at BOTTOM (inverting  
mode)  
        // and 2 + 1 for selecting Fast PWM, 10-bit  
    TCCR1B = 4 + 8; // 4 for selecting 256 prescaler & 8  
for selecting Fast PWM, 10-bit  
    OCR1B = 1000;  
    TCNT1 = 0;  
  
    while(1){}  
}  
  
void loop() {}
```

08. Timer1 programming in Fast PWM (10-bit Non - Inverting) Mode for OC1B pin:-

```
void setup() {  
  
    DDRB = 4; // PORTB 2nd bit set as Output  
  
    TCCR1A = 32 + 2 + 1; // 32 for clear OC1B on  
compare match, set OC1B at BOTTOM (Non-inverting  
mode)  
        // and 2 + 1 for selecting Fast PWM, 10-bit  
    TCCR1B = 4 + 8; // 4 for selecting 256 prescaler & 8  
for selecting Fast PWM, 10-bit  
    OCR1B = 10;  
    TCNT1 = 0;  
  
    while(1){}  
}  
  
void loop() {}
```

09. The High-lighted Modes are not done for the confusion :-

Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1x at	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, phase correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, phase correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, phase correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, phase and frequency correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, phase and frequency correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, phase correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, phase correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	(Reserved)	–	–	–
14	1	1	1	0	Fast PWM	ICR1	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCR1A	BOTTOM	TOP

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	WGM13:0 = 9 or 11: Toggle OC1A on compare match, OC1B disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on compare match when up-counting. Set OC1A/OC1B on compare match when down counting.
1	1	Set OC1A/OC1B on compare match when up-counting. Clear OC1A/OC1B on compare match when down counting.

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	WGM13:0 = 14 or 15: Toggle OC1A on compare match, OC1B disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on compare match, set OC1A/OC1B at BOTTOM (non-inverting mode)
1	1	Set OC1A/OC1B on compare match, clear OC1A/OC1B at BOTTOM (inverting mode)

11. Timer1 programming in Phase Correct PWM Mode [10-bit] (COM0A1=1 & COM0A0=0) for OC1A pin:-

```
void setup() {
```

```
    DDRB = 2; // PORTB 1st bit set as Output
```

```
    TCCR1A = 128 + 2 + 1; // 128 for clear OC1A on
    compare match when up-counting
    // & set OC1A on compare match when
    down counting.
```

```
    // and 2 + 1 for selecting PWM, phase
    correct, 10-bit
```

```
    TCCR1B = 4 ; // 4 for selecting 256 prescaler
```

```
    OCR1A = 10;
```

```
    TCNT1 = 0;
```

```
    while(1){}
}
```

```
void loop() {}
```

10. Timer1 programming in Phase Correct PWM Mode [10-bit] (COM0A1=1 & COM0A0=1) for OC1A pin:-

```
void setup() {
```

```
    DDRB = 2; // PORTB 1st bit set as Output
```

```
    TCCR1A = 128 + 64 + 2 + 1; // 128 + 64 for set OC1A
    on compare match when up-counting
    // & clear OC1A on compare match
    when down counting.
```

```
    // and 2 + 1 for selecting PWM,
    phase correct, 10-bit
```

```
    TCCR1B = 4 ; // 4 for selecting 256 prescaler
```

```
    OCR1A = 1000;
```

```
    TCNT1 = 0;
```

```
    while(1){}
}
```

```
void loop() {}
```

12. Timer1 programming in Phase Correct PWM Mode [10-bit] (COM1B1=1 & COM1B0=1) for OC1B pin:-

```
void setup() {
```

```
    DDRB = 4; // PORTB 2nd bit set as Output
```

```
    TCCR1A = 32 + 16 + 2 + 1; // 32 + 16 for set OC1B
    on compare match when up-counting
    // & clear OC1B on compare match
    when down counting.
```

```
    // and 2 + 1 for selecting PWM, phase
    correct, 10-bit
```

```
    TCCR1B = 4 ; // 4 for selecting 256 prescaler
```

```
    OCR1B = 1000;
```

```
    TCNT1 = 0;
```

```
    while(1){}
}
```

```
void loop() {}
```

13. Timer1 programming in Phase Correct PWM Mode [10-bit] (COM1B1=1 & COM1B0=0) for OC1B pin:-

```
void setup() {  
  
    DDRB = 4; // PORTB 2nd bit set as Output  
  
    TCCR1A = 32 + 2 + 1; // 32 for clear OC1B on  
compare match when up-counting  
        // & set OC1B on compare match when  
down counting.  
        // and 2 + 1 for selecting PWM, phase  
correct, 10-bit  
    TCCR1B = 4 ; // 4 for selecting 256 prescaler  
    OCR1B = 10;  
    TCNT1 = 0;  
  
    while(1){}  
}  
  
void loop() {}
```

14. Timer1 programming, CTC Mode Compare Match A:-

```
void setup() {  
  
    DDRB = 2; // PORTB 1st bit set as Output  
  
    TCCR1A = 0; // TCCR1A = 0 & TCCR1B = 8 for CTC  
Mode  
    TCCR1B = 8 + 4; // 4 for 256 prescaler  
    TIMSK1 = 2; // for selecting the Output Compare  
Match A Interrupt  
    OCR1A = 60000;  
    TCNT1 = 0;  
  
    while(1){}  
}  
  
ISR(TIMER1_COMPA_vect){  
    PORTB ^= 2;  
}  
  
void loop() {}
```

15. Timer1 programming, CTC Mode Compare Match B:-

```
void setup() {  
  
    DDRB = 2; // PORTB 1st bit set as Output  
  
    TCCR1A = 0; // TCCR1A = 0 & TCCR1B = 8 for CTC  
Mode  
    TCCR1B = 8 + 4; // 4 for 256 prescaler  
    TIMSK1 = 4; // for selecting the Output Compare  
Match B Interrupt  
    OCR1A = 60000;  
    TCNT1 = 0;  
  
    while(1){}  
}  
  
ISR(TIMER1_COMPB_vect){  
    PORTB ^= 2;  
}  
  
void loop() {}
```

16. Use of TIFR Register (self) :-

```
void setup() {  
  
    DDRB = 2; // PORTB 1st bit set as Output  
  
    TCCR1A = 0; // TCCR1A = 0 & TCCR1B = 8 for CTC  
Mode  
    TCCR1B = 8 + 4; // 4 for 256 prescaler  
    TIMSK1 = 2; // for selecting the Output Compare  
Match A Interrupt  
    OCR1A = 60000;  
    TCNT1 = 0;  
  
    while(1){}  
}  
  
ISR(TIMER1_COMPA_vect){  
    if((TIFR1 & 2)==0)  
        PORTB ^= 2;  
}  
  
void loop() {}
```

